# On The Conservation Laws and Pathway Realizability of Biochemical Reaction Networks

Thanh-An Pham[1] and Tieu-Long Phan[1,2*]

[1]Bioinformatics Group, Department of Computer Science & Interdisciplinary Center for Bioinformatics & School for Embedded and Composite Artificial Intelligence (SECAI), Leipzig University, Härtelstraße 16–18, D-04107, Leipzig, Germany.
[2]Department of Mathematics and Computer Science, University of Southern Denmark, DK-5230, Odense M, Denmark.

*Corresponding author(s). E-mail(s): tieu@bioinf.uni-leipzig.de;
Contributing authors: thanh-an.pham@uni-leipzig.de;

**Abstract**

**Background:** Conserved metabolite pools and substrate availability constrain biochemical pathways before kinetic parameters are known. Analyzing these constraints across curated and rule-generated networks requires both reaction stoichiometry and the amounts of substrates consumed at each step. Species that appear on both sides of a reaction cancel from the net stoichiometry, although the reaction still needs them to fire.

**Results:** We present the chemical reaction network (CRN) submodule of `SynKit`, an open-source Python toolkit, for constructing biochemical reaction networks and analyzing their conserved pools, chemical reaction network theory (CRNT) properties, and realizable reaction sequences. `KEGG` pathways, supported `SBML` models, and rule-generated reactions use the same reaction records. Twelve reference networks pass the validation checks and agree with `CRNT4SBML` on eight shared structural quantities. Ten reaction-bearing `BioModels` entries pass structural `SBML` round trips, including a model with 504 species and 827 directed reactions. Four `KEGG` modules illustrate conserved pools and the effects of model boundaries. In glycolysis, removing shared polyphosphate from a reaction changes deficiency and substrate requirements while preserving the net stoichiometric matrix. A 15-step glucose-to-pyruvate firing sequence is realizable after currency metabolites are removed but is blocked from glucose alone when they are retained. In a generated formose network, conservation coefficients match carbon counts for all 37 species, but equal carbon inventories differ in their ability to supply the selected tetrose and hexose routes. Native rule expansion that keeps hydrogens as per-atom counts reproduces this network exactly and conserves atoms and charge in every reaction of three hydrogen-transfer controls, where re-deriving hydrogens from default valences does not. C++ implementations complete all reported structural enumerations, including all 60 configurations of the scaling study.

**Conclusions:** `SynKit` brings conservation analysis and reaction-by-reaction realizability checks to the same biochemical model. Its results help distinguish conserved quantities, unmet substrate requirements, and the assumptions needed for CRNT conclusions. Source code, recorded results, and reproducible analyses accompany the manuscript.

**Keywords:** Chemical reaction networks, chemical reaction network theory, Petri nets, conserved moieties, stoichiometry, pathway realizability, `SBML`

## 1 Background

Curated databases such as `KEGG` [1], `Reactome` [2], and `WikiPathways` [3] provide biochemical reaction records for reuse, while graph-transformation methods enumerate candidate reactions from molecular seeds and transformation rules [4]. Before assigning kinetic parameters, researchers need to determine which quantities a network conserves, which structural theorems apply, and whether a proposed route can fire from the available substrates. Answering these questions on the same reaction system is useful for checking metabolic modules and screening generated reaction spaces.

These analyses require different information. A species graph represents connectivity but can lose the joint requirement of multiple substrates [5]. The net stoichiometric matrix records changes in species amounts but cancels species present on both sides. Over the same species set, $A + C \to B + C$ and $A \to B$ have identical net columns, yet only the first needs $C$ to fire, and the two have different complexes. Similarly, a balanced reaction combination or a qualitatively reachable product does not establish that the reactions can fire in an admissible order from a finite inventory [6]. Conversion choices can therefore preserve one analysis while changing another.

Existing packages address subsets of these tasks. `COBRApy` [7] supports metabolic reconstruction and constraint-based flux analysis, including flux balance analysis [8] and elementary flux modes [9]. `CRNT4SBML` [10] provides deficiency analysis and bistability detection. `MØD` [4] combines graph rewriting with reaction-space exploration, and `Tellurium` [11] and `libRoadRunner` [12] support kinetic simulation. Petri-net representations of metabolic networks are widely used in structural analysis [13, 14] and provide pathway realizability certificates [6]. `SBML` [15] and `COMBINE` [16] standardize model exchange, while converters such as `KEGGtranslator` [17] export curated pathways into `SBML`. However, file conversion alone does not guarantee that the structural assumptions required for each analysis are preserved.

Conserved pools and substrate sufficiency need to be examined on the same imported or generated reaction list. A glycolytic route may appear connected from glucose to pyruvate when ATP and redox cofactors are ignored, yet fail to fire when those cofactors are retained as required reactants with limited initial supplies. In a generated carbohydrate network, every reaction may conserve carbon, yet a proposed pathway may remain unrealizable because its reactions require more glycolaldehyde molecules than the initial inventory contains. In both cases, network structure and conservation laws alone do not establish that the sequence can run from the specified finite inventory.

We developed the chemical reaction network (CRN) submodule of `SynKit` [18] to run these checks on a shared reaction record. It connects `KEGG` ingestion, structural `SBML` exchange, and rule-based construction to exact conservation and deficiency calculations, Petri-net siphon analysis, and replayable firing certificates, while reporting the hypotheses required by each conclusion. We validate the submodule on twelve reference networks, compare eight shared structural quantities against `CRNT4SBML`, test structural round trips on ten `BioModels` entries spanning two orders of magnitude in size, and measure the cost of exact enumeration. Controlled experiments in glycolysis and in a rule-generated formose network then isolate how participant cancellation, cofactor boundaries, and precursor mixtures alter deficiency and pathway realizability even when net stoichiometry or total carbon is unchanged.

# 2 Implementation

## 2.1 Reaction-network construction and representation

Within the CRN submodule, `SynCRN` stores species, reactions, rules, and source annotations as reaction records. It derives a `NetworkX`-compatible bipartite digraph, stoichiometric matrices, a Petri net, and reaction equations on demand (Figure 1). The graph partitions species $X$ and reactions $\mathscr{R}$, where species-to-reaction and reaction-to-species edges encode consumption and production. Reading each reaction and its incident edges as a directed hyperedge preserves multi-substrate dependencies.

For reaction $r$, the reactant and product multisets have coefficients $s_{xr}^{-}$ and $s_{xr}^{+}$. The integer-stoichiometry analyses in this paper use

$$\mathbf{S}^{-} = (s_{xr}^{-}), \quad \mathbf{S}^{+} = (s_{xr}^{+}), \quad \mathbf{S} = \mathbf{S}^{+} - \mathbf{S}^{-}. \tag{1}$$

Shared participants remain in both incidence matrices. Cancelling equal coefficients preserves $\mathbf{S}$ but can change reaction enabling and the complex graph. In Petri-net terms, a species on both sides of a reaction forms a self-loop, and the incidence matrix of a net with self-loops (an impure net) does not determine when its transitions are enabled [14]. Construction does not establish elemental balance automatically, so chemical interpretation requires suitable input equations and annotations, and the conditions under which a reaction network is chemically realizable are themselves a subject of study [19].

The `Query` adapter parses `KEGG` equations, `Construct` expands seed molecules using `SMARTS` or double-pushout graph rules, and `IO` imports supported `SBML` Core reaction structure. Reversible equations can become two directed reactions. Omitting currency metabolites, such as ATP, NADH, water, or protons, or excluding boundary species changes the analyzed system. These choices are specified when constructing the network. The `Symmetry` module combines Weisfeiler–Lehman color refinement [20] with exact graph methods for canonicalization, isomorphism, and automorphism analysis [21, 22]. Automorphisms reflect graph structure unless species labels are fixed during matching.

## 2.2 Exact stoichiometric analysis and conserved pools

The stoichiometric matrix $\mathbf{S}$ records the net change in every species for each reaction. For concentrations $c$ and reaction rates $v(c)$, the dynamics are written as $\dot{c} = \mathbf{S}v(c)$. A vector $y$ satisfying $y^{T}\mathbf{S} = 0$ defines a conserved linear quantity $y^{T}c$, because the reactions can redistribute the participating species without changing this weighted total. `SynKit CRN` computes a rational basis for the left nullspace of $\mathbf{S}$ and converts its vectors to primitive integer representatives. These vectors describe all rational conservation relations, but they need not be non-negative or identify the smallest conserved pools.

Minimal P-semiflows are computed separately by exact Farkas elimination [23]. They are non-zero, non-negative integer vectors in the left kernel of $\mathbf{S}$ with inclusion-minimal supports, normalized by their greatest

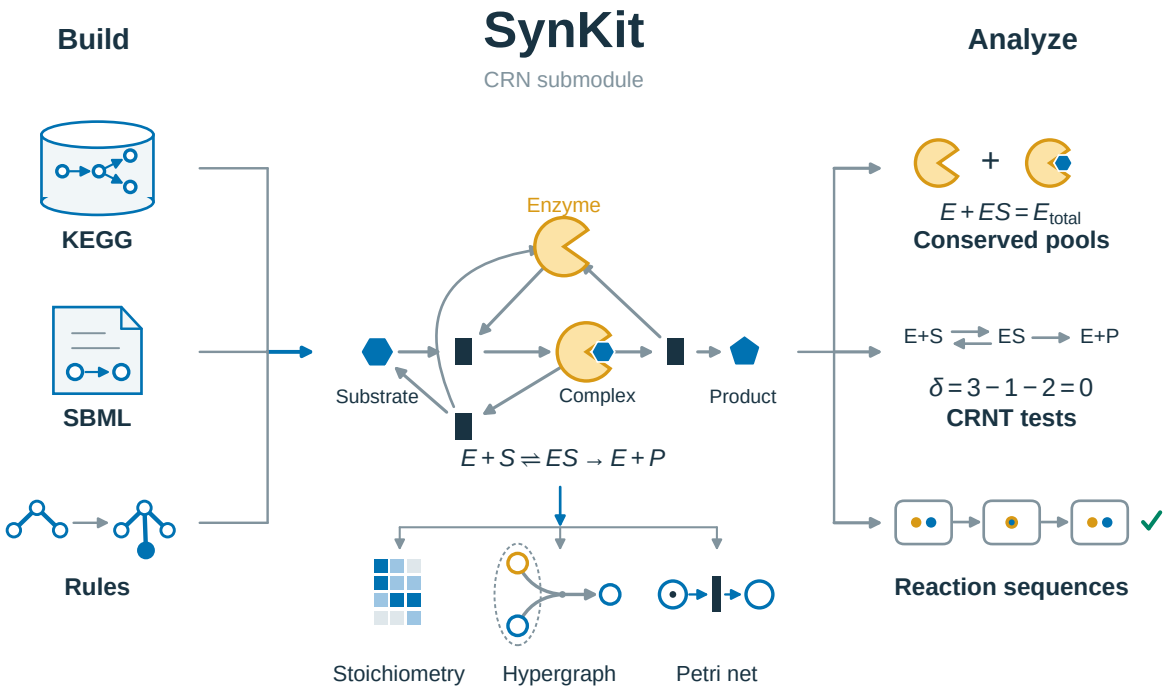



**Fig. 1**: Reaction representation and structural analysis workflows in `SynKit CRN`. Curated pathways (`KEGG`), models (`SBML`), and reaction rules map to a shared bipartite representation. An illustrative enzyme scheme ($E + S \rightleftharpoons ES \to E + P$, with conserved pool $E + ES$) yields stoichiometric, hypergraph, and Petri-net views for exact conservation analysis, CRNT theorem checks, and discrete pathway realizability. Circles denote species and dark bars reactions. The hypergraph view shows one directed hyperedge whose tail set $\{E, S\}$ is consumed jointly. The deficiency and the firing sequence ($E + S \to ES \to E + P$) shown on the right are those of the example. Molecular and diagram icons are schematic. Blue marks species, orange the enzyme, and green a verified result. The other figures are printed in black and white.

common divisor. Their supports identify minimal conserved pools of the represented network [24]. A P-semiflow guarantees mathematical conservation, whereas assigning its support to a chemical moiety requires species annotations and biochemical interpretation.

T-semiflows satisfy $\mathbf{S}x = 0$ with $x \geq 0$ and describe balanced combinations of reactions whose total net species change is zero. They are reaction-space relations, not conserved species pools. A T-semiflow fixes neither a firing order nor an initial marking, and it does not imply thermodynamic feasibility. All-positive left and right kernel vectors define conservativity and consistency, respectively. For positive-kernel feasibility, a numerical linear program proposes a witness, which is accepted only after exact integer or rational verification. Verified positive answers include an all-positive kernel vector, negative answers include a rational separating vector, and unverified solver outcomes remain undecided. Numerical kernel and Jacobian-pattern checks are also available. These describe approximations or possible matrix entries. In particular, a floating-point nullspace basis can represent a subspace accurately without identifying integer minimal supports, so naively rounding such a basis is not a substitute for exact conservation-law computation.

## 2.3 Network deficiency and steady-state theorem applicability

Complexes are distinct reactant or product multisets. With $n$ complexes, $\ell$ connected components of the complex graph (linkage classes), and stoichiometric rank $s$, deficiency is $\delta = n - \ell - s$. A network is weakly

reversible when every linkage class is strongly connected. `SynKit CRN` calculates ranks exactly over $\mathbb{Q}$ and reports the deficiency of the network and of each linkage class, weak reversibility, and the terminal strong linkage classes.

For positive mass-action rate constants, a weakly reversible deficiency-zero network has exactly one positive steady state in each positive stoichiometric compatibility class $(c_0 + \operatorname{im} \mathbf{S}) \cap \mathbb{R}^X_{>0}$, and it is locally asymptotically stable relative to that class [25, 26]. A deficiency-zero network that is not weakly reversible has no positive steady state. The Deficiency One Theorem gives at most one positive steady state per positive stoichiometric compatibility class when each linkage-class deficiency is at most one, their sum equals $\delta$, and each linkage class has one terminal strong linkage class, while weak reversibility additionally ensures existence [27]. Failure of these hypotheses leaves the corresponding theorem undecided. Introductions to this body of theory for biologists are available [28]. Criteria that reach beyond deficiency, including injectivity [29], concordance [30], and graph-theoretic multistationarity tests [31], are not implemented here, and a network left undecided by the deficiency theorems is not thereby shown to be multistationary. The Edelstein example in Supplementary Figure S5 illustrates this distinction.

## 2.4 Petri-net analysis and discrete pathway realizability

A siphon is a non-empty species set for which every reaction producing a member also consumes a member. Once empty, it cannot refill under Petri-net firing semantics. A trap has the dual property that every reaction consuming a member also produces a member, so a marked trap cannot become empty [14]. `SynKit CRN` enumerates minimal siphons using implication propagation and closure minimization, within the branch-and-bound approach to structural enumeration [32]. Producing a selected species forces selection of a reactant. Selected and excluded species are propagated before branching. Each reported minimal set excludes its supersets from further search, and a maximal-closure deletion test verifies minimality. Traps use the dual incidence. This prunes many subsets but does not remove worst-case combinatorial growth.

The package checks whether every minimal siphon contains the support of a P-semiflow, and we call such a siphon covered. Under the boundedness and kinetic assumptions of the corresponding persistence result, this provides a sufficient persistence certificate for positive initial conditions [33]. An uncovered siphon means the test is insufficient, and does not prove extinction or identify a unique nutrient supplementation strategy. The returned details expose these uncovered sets.

A marking $m$ specifies the non-negative integer inventory of each species. Reaction $r$ is enabled precisely when $m_x \geq s^-_{xr}$ for every species, and firing gives $m' = m + \mathbf{S}_{\cdot r}$. Qualitative reachability instead accumulates reachable species sets and ignores consumption and multiplicity. Given a requested flux $f \in \mathbb{N}_0^{|\mathscr{R}|}$, realizability search seeks an enabled sequence firing each reaction exactly $f_r$ times [6]. A positive result includes a certificate consisting of an ordered reaction list that can be checked by updating the inventory after every step. An exhaustive negative result is `unrealizable`, while reaching a state or depth limit is `inconclusive`. Certificates establish firing in the specified discrete model only.

## 2.5 Structural SBML exchange and Python interface

The SBML Level 3 Version 2 Core adapter uses the Python standard library. It preserves supported species metadata, directed reaction stoichiometry, labels, and source identifiers on structural round trips. It does not preserve kinetic laws, events, rules, or package-specific extensions. Logical SBML-qual inputs without Core reactions are diagnosed as reaction-free. Exported SBML structure can be used by compatible downstream tools, with their additional kinetic or flux-bound requirements supplied separately.

The Python interface dispatches exact rational rank and kernel calculations, CRNT invariants, minimal semiflows, siphon and trap enumeration, finite-inventory search, and graph symmetry to a C++17 library compiled before analysis. Arbitrary-precision integer arithmetic uses Boost.Multiprecision. Rule expansion from SMARTS templates also runs natively, in a separate C++ program linked against the RDKit C++ library and kept running for the length of an analysis session. Following the strategy framework of MØD [34], the program keeps a universe of canonical molecules and a subset of newly produced ones, and every rule application must use at least one new molecule. Each molecule is tested once against each reactant template of each rule, and only mixtures that fill every template reach the reaction engine. Hydrogens are kept as counts on heavy atoms, as in the counting projection of Lewis-formula graphs [35]. A mapped hydrogen that a rule moves becomes a decrement on its donor atom and an increment on its acceptor, product hydrogen counts are set from the reactant counts, and an update that leaves a fractional or negative number of lone pairs is rejected. Rules whose hydrogens cannot be compiled this way run with explicit hydrogen vertices, and the output records why. Parsing, SBML exchange, and reporting remain in Python. All production analyses reported here use the C++ backend. The reference checks retain their separate Python, NumPy, SciPy, and NetworkX calculations. The library is compiled with -O3, and the evidence records its build hash, capabilities, and invoked native operations. Searches expose time, state, node, or result limits rather than treating an incomplete answer as complete.

Listing 1 demonstrates the Python interface to the C++ backend using the glycolysis fixture from the full source checkout. ATP/ADP/AMP, NAD(P)(H), water, phosphate, diphosphate, and protons are omitted in this example. The resulting certificate therefore concerns a reduced model. Supplementary Section 1 describes the software architecture and additional analyses.

## 2.6 Validation design and computational benchmarks

Twelve reference networks provide expected structural values and regression cases. Alternative calculations check deficiency through $\operatorname{rank}(\mathbf{I}_a) - \operatorname{rank}(\mathbf{Y}\mathbf{I}_a)$, where $\mathbf{Y}$ is complex composition and $\mathbf{I}_a$ is complex-graph incidence. Numerical singular-value rank provides a further check on these small integer fixtures. Conservation vectors are checked for kernel membership, integrality, and expected count, and semiflows for non-negativity, kernel membership, and absence of nested reported supports. Exhaustive subset enumeration checks minimal siphons. A separate Python implication search agrees with uncapped C++ enumeration

Listing 1: Python analysis of reduced KEGG glycolysis (M00001): network construction, exact structural analysis, discrete pathway verification, and SBML export.

```
from synkit.CRN import (build_native, native_backend, crnt_summary,
    conserved_moieties, find_siphons, siphon_persistence_details,
    PathwayRealizability, crn_to_sbml)
from Experiment.CRN.benchmarks import build_kegg_crn, GLYCOLYSIS_FLUX

library = build_native("/tmp/synkit-crn", with_semiflows=True)
with native_backend(library):
    crn = build_kegg_crn("M00001", drop_currency=True)
    crn.n_species, crn.n_reactions                    # (14, 15)
    report = crnt_summary(crn)
    report.deficiency, report.is_weakly_reversible    # (1, False)
    len(conserved_moieties(crn)), len(find_siphons(crn)) # (3, 3)
    siphon_persistence_details(crn).persistence_ok    # False

    check = PathwayRealizability().load_syncrn_and_flow(
        crn, flow=GLYCOLYSIS_FLUX,
        initial_marking={"alpha-D-Glucose": 1},
        species="label", reaction="label")
    check.build_petri_net_from_flow()
    outcome = check.realizability_result()
    outcome.status.value, len(outcome.certificate) # ("realizable", 15)
    sbml_document = crn_to_sbml(crn)
```

on the full siphon sets across all 31 tested networks (twelve references, four random instances, ten reaction-bearing BioModels entries, four KEGG modules, and the formose network). Direct incidence checks verify every returned set and its minimality. These routes share input parsing and some representation helpers, and the implication searches share a search strategy, so they are not wholly independent implementations. Checks on returned vectors alone do not establish enumeration completeness. An external CRNT4SBML comparison supplies an independent parser and implementation for eight shared quantities.

The larger scaling study uses three fixed network families and one timing per configuration, with a 300-second wall-clock budget and a 200 000-evaluation budget for siphon search. A separate paired experiment repeats both C++ siphon algorithms five times at 11, 15, 19, and 21 species, alternates execution order, uses fresh objects, excludes network construction time, and records every sample plus medians and quartiles. Python interface costs remain in the measurements. The representation experiment keeps species identities fixed while cancelling common reactant/product coefficients in M00001, then compares inventories with and without the shared polyphosphate species. A second control retains or removes currency metabolites under the same glucose-only marking and requested flux. Successful certificates are independently replayed against reaction records. Every reported enumeration must complete for the study to pass. Positive discrete decisions require verified certificates, and negative decisions require exhaustive search of the specified finite problem. All studies record checks and input sources in machine-readable reports.

# 3 Results and discussion

## 3.1 Reference-network validation and comparison with CRNT4SBML

All twelve reference networks pass their expected-value and algebraic checks (Table 1). The set includes enzyme mechanisms, phosphorylation cycles [36], the Edelstein switch [37], the Shinar–Feinberg motif [38],

disconnected networks, and an open reactor. Disconnected reversible pairs test separation of minimal conservation supports, whereas the open reactor tests inclusion of the zero complex. These are targeted correctness examples rather than a representative sample of biological networks.

`CRNT4SBML` 0.0.15 independently reads the twelve exported `SBML` networks. Species, directed reactions, complexes, linkage classes, rank, deficiency, weak reversibility, and linkage-class deficiencies agree in every case (Table 1). The isolated comparator environment uses Python 3.7.12, `libSBML` 5.18.0, and `NetworkX` 2.3. This comparison supports agreement on shared structural outputs, but it does not benchmark relative speed or compare bistability-search capabilities. A supplementary numerical audit finds small floating-point nullspace residuals on all twelve fixtures, while naive scaling and integer rounding produces invalid vectors on 3 networks (Supplementary Table S8). Exact integer vectors satisfy the identity in every case. This supports exact reconstruction for integer invariants without claiming that numerical subspace estimation is intrinsically inaccurate.

**Table 1**: Structural invariants of the twelve reference networks: species, directed reactions, complexes ($n$), linkage classes ($\ell$), stoichiometric rank ($s$), deficiency ($\delta$), conservation dimension (laws), and weak reversibility (WR). Every network passes its expected-value and algebraic checks, and `CRNT4SBML` reproduces all eight shared structural quantities for each network.

| Network | Species | Reactions | $n$ | $\ell$ | $s$ | $\delta$ | Laws | WR |
|---|---|---|---|---|---|---|---|---|
| Reversible isomerization | 2 | 2 | 2 | 1 | 1 | 0 | 1 | yes |
| Irreversible isomerization | 2 | 1 | 2 | 1 | 1 | 0 | 1 | no |
| Isomerization cycle | 3 | 3 | 3 | 1 | 2 | 0 | 1 | yes |
| Disconnected reversible pairs | 6 | 6 | 6 | 3 | 3 | 0 | 3 | yes |
| Pairs with stoichiometry | 6 | 6 | 6 | 3 | 3 | 0 | 3 | yes |
| Michaelis–Menten | 4 | 3 | 3 | 1 | 2 | 0 | 2 | no |
| Reversible Michaelis–Menten | 4 | 4 | 3 | 1 | 2 | 0 | 2 | yes |
| Edelstein switch | 3 | 6 | 5 | 2 | 2 | 1 | 1 | yes |
| Phosphorylation: one site | 6 | 6 | 6 | 2 | 3 | 1 | 3 | no |
| Phosphorylation: two sites | 9 | 12 | 10 | 2 | 6 | 2 | 3 | no |
| Shinar–Feinberg motif | 2 | 2 | 4 | 2 | 1 | 1 | 1 | no |
| Open reactor | 2 | 3 | 3 | 1 | 2 | 0 | 0 | yes |

## 3.2 Computational cost of exact structural enumeration

Figure 3b reports the repeated paired C++ experiment. Both algorithms return identical minimal siphons for every run. At 21 species, median exhaustive-search time is 55.80 ms and median pruned-search time is 1.47 ms. The ratio of median times changes from 1.2 at 11 species to 38.1 at 21 species. The methods have comparable times at the smallest size, while exhaustive search becomes more expensive as the subset space grows. Quartiles describe repeat-to-repeat variation on these fixed networks, rather than variation across network topologies, and four sizes on one family do not establish asymptotic complexity.

The larger C++ sweep includes directed chains, reversible chains, and random sparse bimolecular networks (Supplementary Figure S6). All 60 requested configurations complete. On the 401-species reversible chain, minimal siphons take 0.445 s, canonicalization 0.495 s, exact conserved pools 1.78 s, and standalone exact rank 2.8 s (Figure 3c). The random-family networks with 49, 99, 197, and 395 species have 331, 272, 47, and 87 minimal siphons, respectively. At 395 species, siphon enumeration completes in 0.161 s and conserved-pool enumeration in 0.0607 s. These single-run measurements depend on topology, algorithm, and

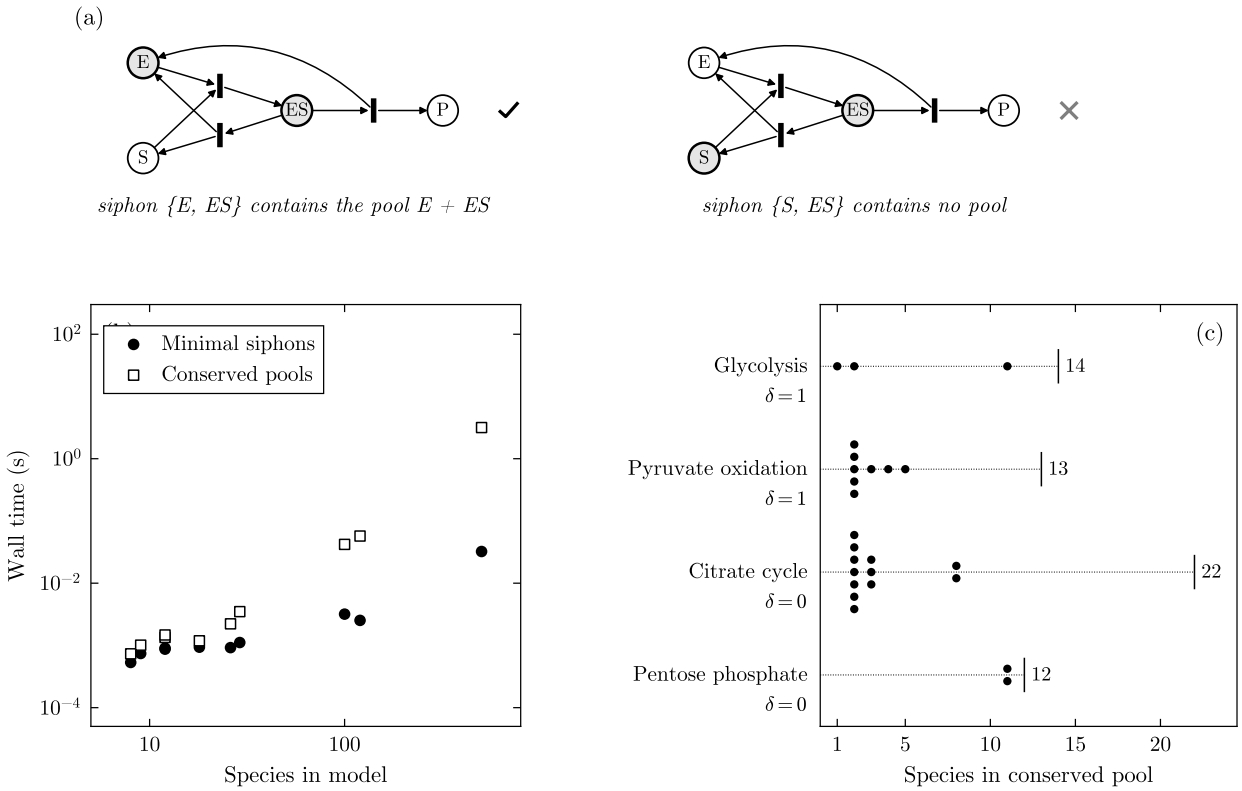



**Fig. 2**: Siphons and conserved pools, with C++ enumeration cost in `BioModels` and conserved pools in `KEGG` modules. **(a)** The enzyme scheme of Figure 1 as a Petri net (circles: species, bars: reactions, shaded: members of a minimal siphon). The siphon $\{E, ES\}$ contains the conserved pool $E + ES$, so it cannot become empty once marked (check). The siphon $\{S, ES\}$ contains no conserved pool and is not certified (cross). **(b)** Wall time for completed minimal-siphon and exact conserved-pool enumeration in ten reaction-bearing models. All ten models complete both enumerations. **(c)** Minimal non-negative conserved pools in four reduced modules. Each dot is one pool placed at its number of species, pools of equal size are stacked, and the dotted line spans the module species, ending at the module size.

implementation. They establish neither a polynomial bound for enumeration nor a universal species-count ceiling.

### 3.3 Structural `SBML` round-trip validation in `BioModels`

A size-stratified manifest selects eleven `BioModels` [39] entries comprising ten reaction-bearing models and one `SBML-qual` negative control. All ten pass the supported structural export/re-import checks, and their deficiency identities hold (Figure 2b). Source identifiers, URLs, and input checksums accompany the results. The largest, `BIOMD0000000255`, contains 504 species and 827 directed reactions. Its C++ enumeration finds 75 minimal siphons and 17 minimal conserved pools in 0.0323 s and 3.17 s, respectively. All ten models complete both enumerations and their siphon-coverage checks; neither `BIOMD0000000255` nor the 100-species `BIOMD0000000019` (20 minimal siphons, 19 minimal conserved pools) satisfies the sufficient persistence condition. `BIOMD0000000562` is correctly diagnosed as reaction-free for Core structural analysis. The selected set tests interoperability over a range of sizes and does not estimate repository-wide support.

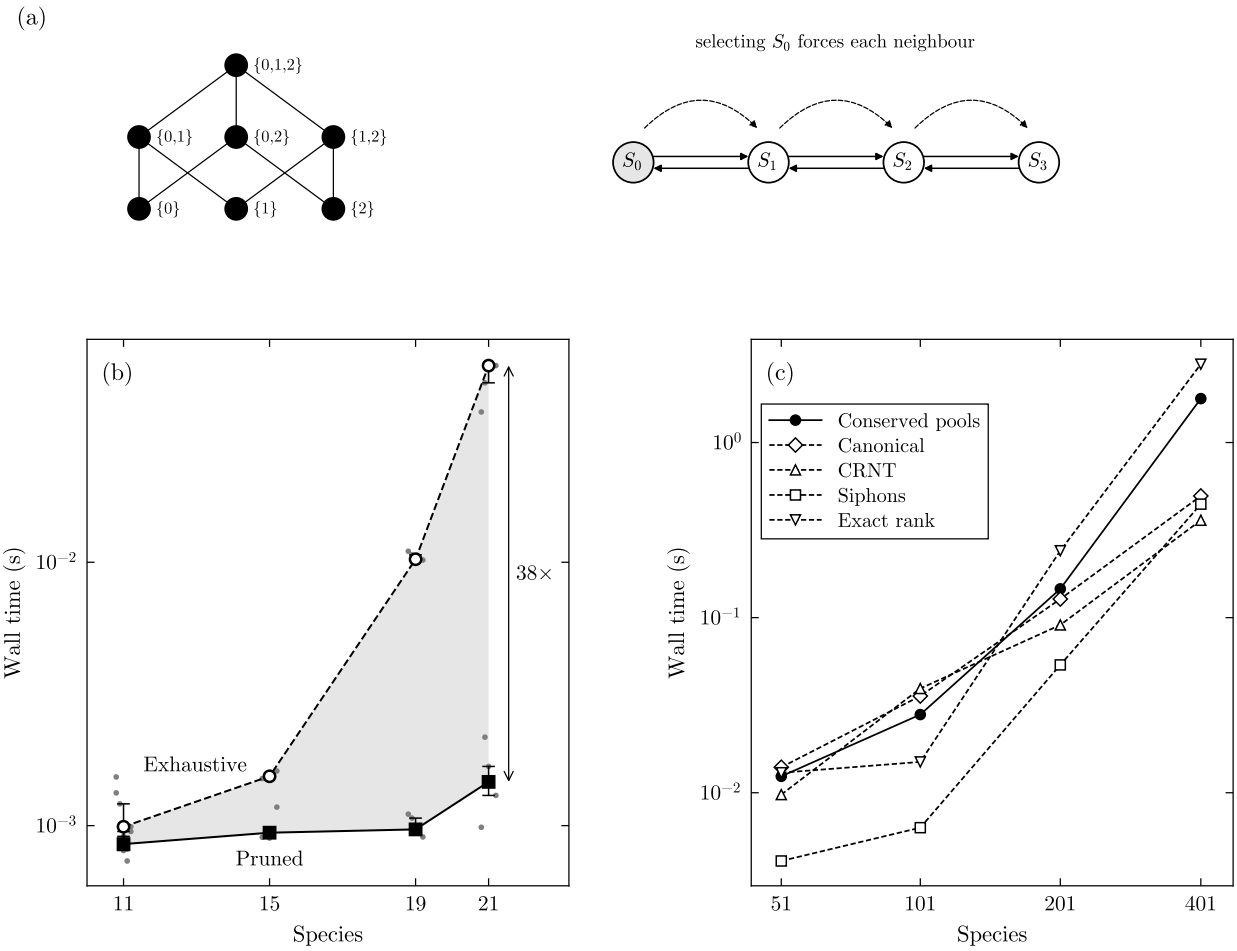



**Fig. 3**: C++ runtime comparison of siphon algorithms and structural analyses on reversible-chain networks. **(a)** Exhaustive search tests every non-empty species subset. Implication pruning selects one species and propagates: every reaction producing a selected species must consume one, so in the reversible chain the selection of $S_0$ forces the whole chain, its only minimal siphon (dashed arrows). **(b)** Exhaustive versus implication-pruned minimal-siphon enumeration across five repeats per size (points show individual runs, markers and error bars show medians and IQRs, and every pair returns identical siphon sets). Shading spans the gap between medians, and the bracket gives their ratio at 21 species. **(c)** Single-run wall times for five analyses across larger reversible chains. Both panels include Python interface overhead, and lines connect successive sizes.

## 3.4 Conservation and cofactor requirements in glycolysis

Four cached `KEGG` [1] modules cover glycolysis (`M00001`), pyruvate oxidation (`M00307`), the citrate cycle (`M00009`), and the pentose phosphate pathway (`M00004`). Their conserved-pool supports appear in Figure 2c. In the reduced citrate-cycle model, minimal non-negative conservation vectors recover nucleotide pairs, a quinone pair, a ferredoxin pair, coenzyme A, and a lipoyl pool. Because these interpretations reflect the retained reaction equations and labels, they do not imply organism-specific physiological pool conservation.

The reduced glycolysis model has 14 species and 15 directed reactions ($n = 16$, $\ell = 4$, $s = 11$, and $\delta = 1$). Its largest minimal conservation support spans eleven carbohydrate intermediates, assigning weight two to hexoses and one to trioses (Figure 4). Glucose and oxidized ferredoxin form uncovered singleton siphons. Polyphosphate forms a covered singleton because the cached generic-polymer reaction retains it on both sides. This symbolic representation does not resolve polymer chain length or prove catalytic phosphate chemistry.

For one glucose token, the prescribed flux has a 15-step firing certificate ending in two pyruvate tokens, and search explores 30 states. The repeated lower-pathway firings are specified in the requested flux and are compatible with the two trioses produced by aldolase. Certificate replay verifies enabling at each step and conservation of the weighted carbohydrate pool. Requesting only the lower pathway from glucose fails because the required intermediates are absent.

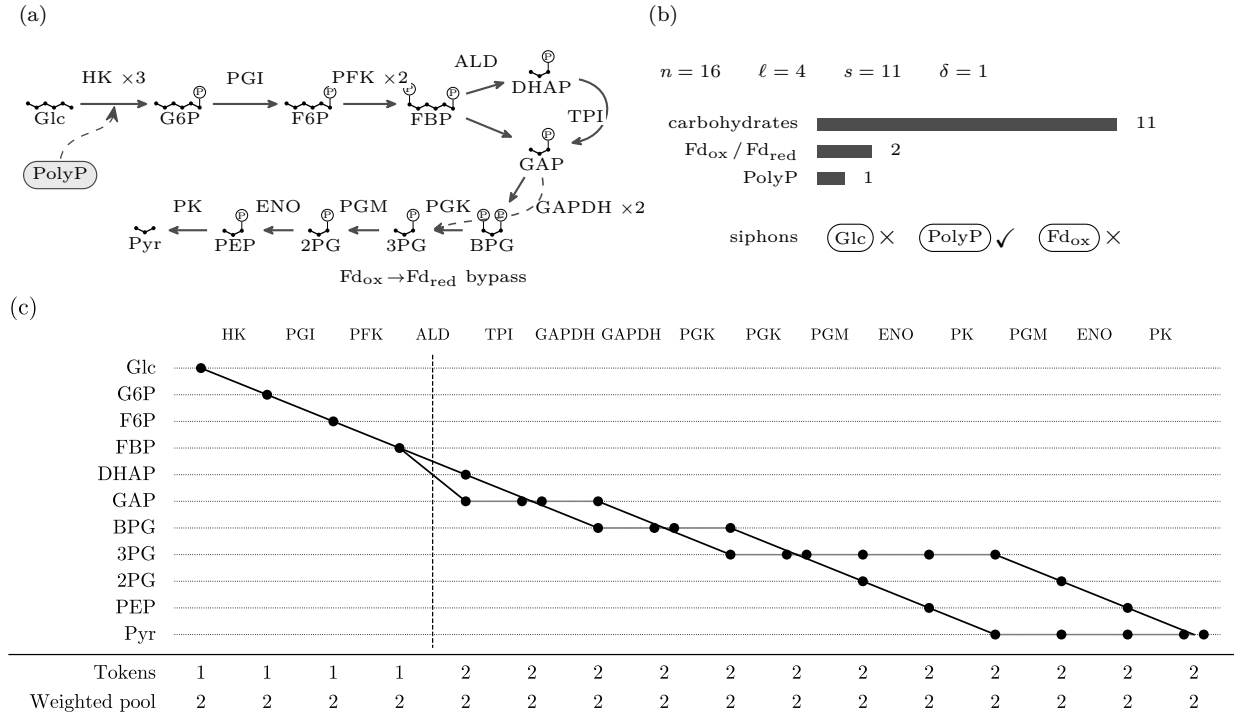



**Fig. 4**: Conserved pools and pathway realizability in reduced KEGG glycolysis (M00001). **(a)** Bipartite network (14 species, 15 directed reactions, currency metabolites omitted) retaining shared polyphosphate and the ferredoxin bypass, where edge labels indicate parallel alternatives. Each metabolite is drawn as its carbon skeleton (dots: carbon atoms, circled P: phosphate), so hexoses carry six carbons and trioses three. **(b)** Structural invariants, conserved pools (bar length: species per pool), and minimal siphons (check: covered by a conserved pool, cross: uncovered). The uncovered singleton siphons, glucose and oxidized ferredoxin, leave persistence uncertified. **(c)** Token markings across the certified 15-step conversion of one glucose to two pyruvates (each dot is one token, lines link each consumed intermediate to its products, column labels name the enzyme fired, and the dashed line marks aldolase cleavage). Bottom rows trace total carbohydrate tokens and the conserved weighted pool ($2 \times \text{hexoses} + 1 \times \text{trioses} = 2$). Inactive polyphosphate and ferredoxin are omitted.

The controlled experiment isolates the information lost by cancellation (Figure 5). Removing the common polyphosphate incidence from R02189 preserves every species row, reaction column, and entry of $\mathbf{S}$. Consequently, its kernel and conservation space are unchanged. Yet the complex count changes from 16 to 14, linkage classes from four to three, and deficiency from one to zero. With one glucose and zero polyphosphate tokens, the retained reaction is blocked, whereas the cancelled reaction fires. Supplying one polyphosphate token enables the retained reaction. Two networks with the same net stoichiometric matrix can therefore differ in deficiency and in which reactions can fire.

A separate boundary control restores the currency metabolites, producing a 24-species model with the same 15 reactions. Under the same glucose-only inventory and requested flux, no reaction is enabled, because hexokinase lacks ATP, and exhaustive search returns `unrealizable`. The 15-step certificate therefore depends on currency removal. The reduced sequence establishes conversion of the carbohydrate intermediates, and evaluating cofactor sufficiency requires restoring the omitted species and specifying their initial amounts. Supplementary Section 2.1 gives a cofactor-coupled example with explicit ATP and ADP alternatives.

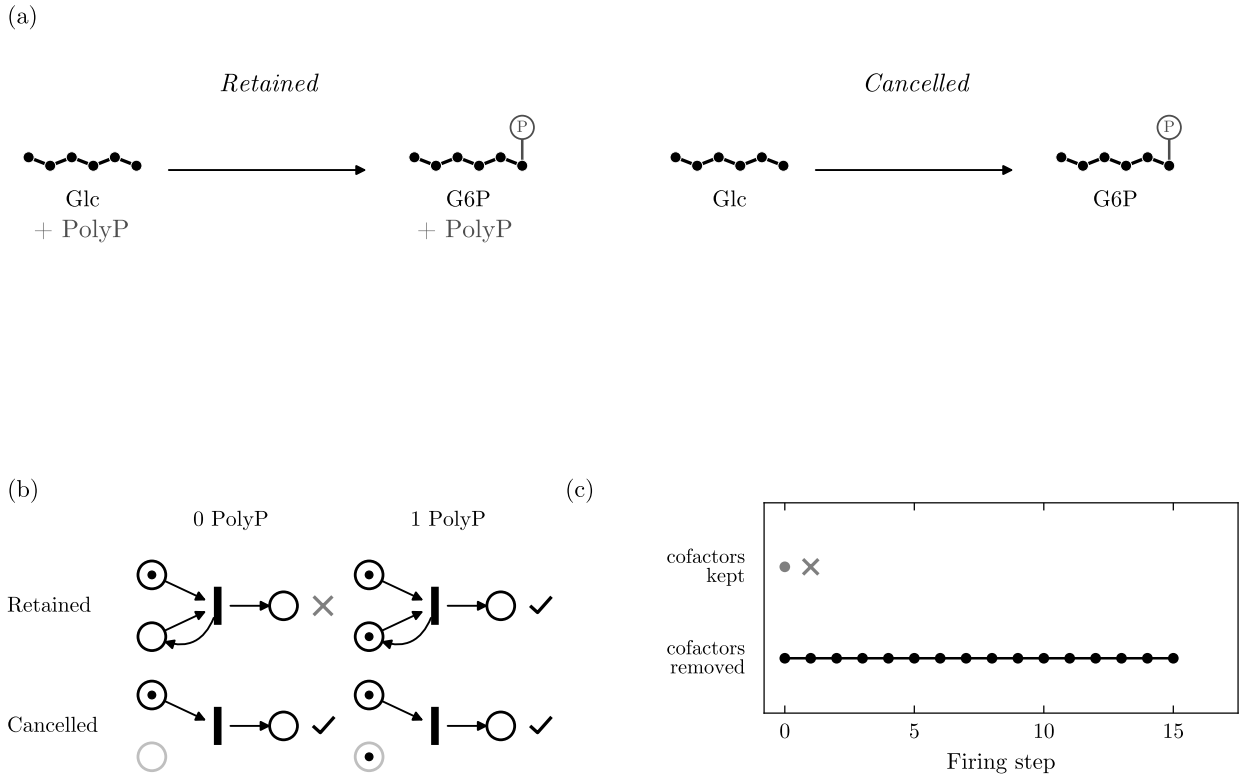



**Fig. 5**: Sensitivity of structural invariants and pathway realizability to participant retention and currency omission. **(a)** Cancelling shared polyphosphate (`PolyP`) in reaction `R02189` preserves net stoichiometry ($\mathbf{S}_{\text{retained}} = \mathbf{S}_{\text{cancelled}}$) but changes complex count ($n : 16 \to 14$), linkage classes ($\ell : 4 \to 3$), and deficiency ($\delta : 1 \to 0$). Glc indicates glucose, G6P indicates glucose 6-phosphate, filled circles represent carbon atoms, and P represents phosphate. **(b)** `R02189` as a Petri net under zero or one `PolyP` token (upper input place: glucose, lower input place: `PolyP`, output place: G6P, dots: tokens). The retained reaction consumes and returns `PolyP`, so it fires only when a `PolyP` token is present (check: enabled, cross: blocked). **(c)** Firing steps of the prescribed glycolytic flux from one glucose. With the currency metabolites (cofactors) kept in the model but not supplied (24 species), hexokinase lacks ATP, no reaction is enabled, and exhaustive search fails. With them removed (14 species), a certified 15-step sequence reaches two pyruvates.

## 3.5 Carbon conservation and precursor requirements in formose networks

The formose reaction, in which formaldehyde condenses to a mixture of sugars [40], gives a rule-generated counterpart to the curated `KEGG` modules. Starting from formaldehyde and glycolaldehyde, four rules for aldol addition, retro-aldol cleavage, and reversible keto-enol tautomerization generate 37 species and 44 reactions after four iterations. Generation takes 0.0144 s in the C++ run and reaches species containing

ten carbon atoms. The iteration count is a rule-expansion depth, not physical reaction time. The network has 54 complexes, 17 linkage classes, rank 36, and deficiency one.

Its one-dimensional left nullspace yields a positive primitive conservation vector. Comparing its coefficients with carbon counts from the molecular graphs gives exact agreement for all 37 species (Figure 6). This is a consistency check between generated stoichiometry and molecular structures. Within this carbohydrate-only rule set, elemental counts can be proportional, but the result does not show that stoichiometry uniquely identifies elemental composition in general.

The hydrogen model of the expansion is a representation choice of the same kind as cancelling shared participants. With explicit hydrogen vertices, every hydrogen of a molecule is matched separately, and the hydrogen counts of product atoms are re-derived from default valences after each rule application. With hydrogens kept as per-atom counts, molecules are matched as heavy-atom graphs and product counts follow from the rule's hydrogen moves. Both models generate the same formose network after four, five, and six iterations. At six iterations (10 572 species and 11 239 reactions), expansion takes 5.9 s with hydrogen counts and 9.0 s with hydrogen vertices. The models differ when a rule's hydrogen bookkeeping is incomplete or involves charged atoms with fixed hydrogen counts (Supplementary Table S4). Re-deriving hydrogens turns a hydrogen-atom abstraction from ethane back into ethane, finds no reaction for a proton transfer from ammonium to acetate, and adds one hydrogen to each of the 12 reactions of an incomplete alkene insertion rule. With hydrogen counts, all 23 reactions of these three controls conserve atoms and charge, and the open valence left by the incomplete rule appears as a radical. Conservation analysis of a generated network therefore depends on hydrogen accounting during generation, before any stoichiometric matrix is formed.

Qualitative search reaches representative tetrose and hexose targets from the two seed species, but realizability distinguishes inventories that share the same total carbon (Figure 6d). With four formaldehyde and one glycolaldehyde molecule, neither selected route is realizable. With two of each precursor, the tetrose route is realizable but the hexose route is not. With three glycolaldehyde molecules, both are. Each of these inventories contains six carbon atoms. The selected routes therefore require suitable precursor multiplicities as well as a sufficient carbon budget. These results concern fixed reaction multisets in the generated network, and they do not establish unrestricted target unreachability or experimental synthetic feasibility. Supplementary Section 2.2 gives the route definitions and initial pools.

### 3.6 Comparison with existing reaction-network software

The methods in the CRN submodule are established and are also available in other packages. What the module adds is that one reaction record reaches all of them, so a conserved pool, a deficiency value, and a firing certificate refer to the same species set, the same treatment of shared participants, and the same boundary choices. Table 2 places this scope alongside the packages closest to the analyses in this paper.

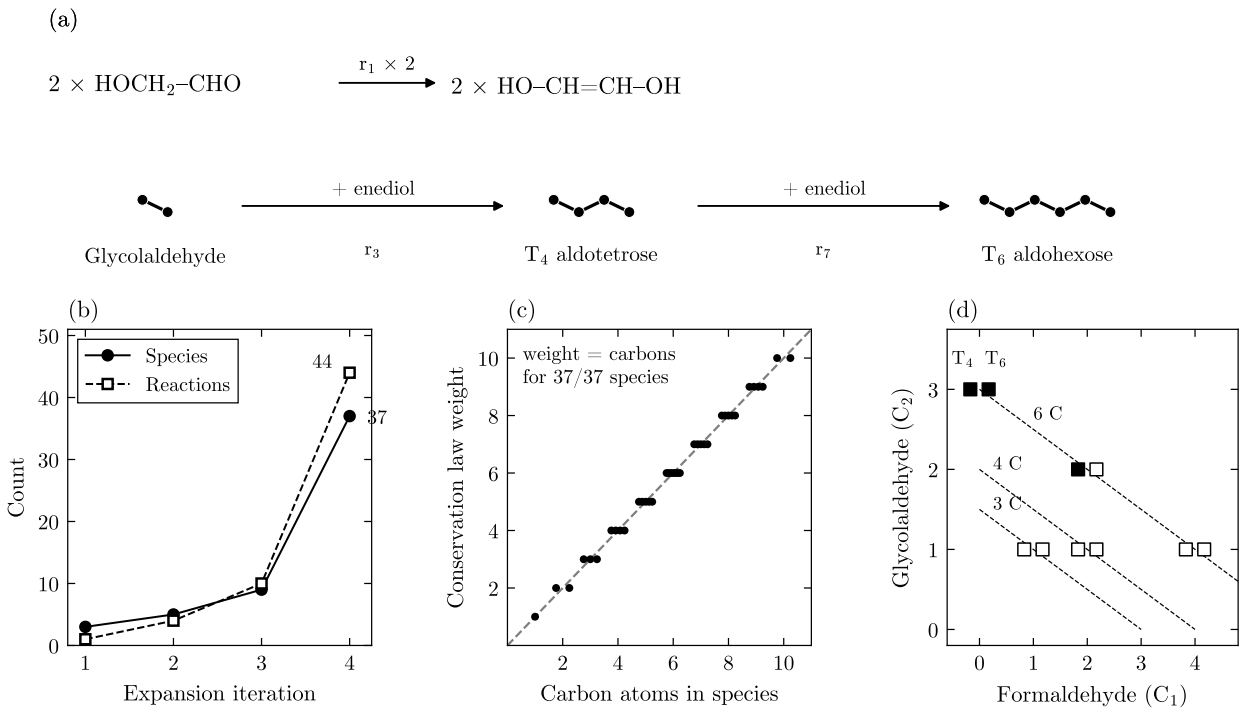



**Fig. 6**: Carbon conservation and precursor-dependent pathway realizability in the generated formose network. **(a)** Target hexose route requiring three glycolaldehyde precursors via two enolizations ($r_1$) and two aldol additions ($r_3$, $r_7$), each addition consuming one enediol (filled circles: carbon atoms). **(b)** Network growth across four rule-expansion iterations from formaldehyde and glycolaldehyde. **(c)** Weights of the single conservation law against carbon atoms per species. They agree for all 37 species, which confirms that the generated reactions conserve carbon (points jittered horizontally, dashed line: $y = x$). **(d)** Five initial inventories by formaldehyde and glycolaldehyde count, with dashed lines joining inventories of equal total carbon. The left and right squares of each inventory refer to the tetrose ($T_4$) and hexose ($T_6$) routes, filled when a certificate is verified and open when exhaustive search fails. Inventories with six carbon atoms differ in which routes they can supply.

Among the packages in Table 2, `crnpy` also provides exact semiflow analysis, and `MolFoundry`, which builds on the `MØD` strategy framework, also generates networks from rules and returns firing orders for integer hyperflows. `SynKit CRN` combines rule-based construction, minimal siphons and traps, exact semiflows, and finite-inventory firing certificates (with exhaustive search for negative answers) in a single object. Bistability search, constraint-based flux analysis, and kinetic simulation require complementary tools such as `CRNT4SBML`, `CoNtRol`, `COBRApy`, and `Tellurium`.

The external evaluation tests agreement with `CRNT4SBML` on shared CRNT outputs rather than comparing usability, end-to-end runtime, or non-overlapping features across packages.

## 3.7 Model assumptions and computational limitations

The conclusions depend on the reaction list, cofactor treatment, reversibility, and boundary species. The glycolysis controls quantify two consequences of changing these assumptions. Minimal-semiflow and siphon enumeration can have combinatorial output and runtime growth. Timeouts and incomplete search must remain visible. None of the analyses predicts kinetic rates, thermodynamic driving forces, or in vivo activity. Future work should prioritize broader input validation and scalable exact enumeration with explicit resource reporting.

**Table 2**: Functional scope of biochemical reaction-network software (✓ documented, (✓) partial or indirect as detailed in footnotes, × not documented). Feature presence is derived from package documentation and source code (for `MolFoundry`, its version 0.1 API documentation), and only `CRNT4SBML` was experimentally evaluated (Table 1). The `SynKit` column describes its CRN submodule.

| | `SynKit` (this work) | `CRNT4SBML` [10] | `CoNtRol` [41] | `crnpy` [42] | `COBRApy` [7] | `MØD` [4] | `MolFoundry` [43] | `Tellurium` [11, 12] |
|---|---|---|---|---|---|---|---|---|
| *Network construction and exchange* | | | | | | | | |
| `KEGG` pathway ingestion | ✓ | × | × | × | × | × | × | × |
| `SBML` import | (✓)[a] | ✓ | × | ✓ | ✓ | × | × | ✓ |
| `SBML` export | (✓)[a] | × | × | ✓ | ✓ | × | ✓ | ✓ |
| Rule-based generation from seeds | ✓ | × | × | × | × | ✓ | ✓ | × |
| *Exact structural analysis* | | | | | | | | |
| Deficiency, linkage classes, weak reversibility | ✓ | ✓ | ✓ | ✓ | × | × | × | × |
| Conservation laws (kernel basis) | ✓ | ✓ | × | ✓ | × | × | × | ✓ |
| Minimal P-semiflows (minimal support) | ✓ | × | × | ✓ | × | × | × | × |
| T-semiflows or elementary modes | ✓ | × | × | ✓ | × | × | × | × |
| Minimal siphons and traps | ✓ | × | (✓)[b] | × | × | × | × | × |
| *Behavior and realizability* | | | | | | | | |
| Multistationarity or bistability search | × | ✓ | ✓ | (✓)[c] | × | × | × | × |
| Finite-inventory firing certificate | ✓ | × | × | × | × | (✓)[d] | ✓[f] | × |
| Constraint-based flux analysis | × | × | × | × | ✓ | × | (✓)[f] | × |
| Kinetic simulation | × | (✓)[e] | × | × | × | × | ✓[g] | ✓ |

[a] `SBML` Level 3 Version 2 Core reaction structure and metadata only, as kinetic laws, events, rules, and package extensions are not handled.
[b] Siphons underlie `CoNtRol`'s persistence conditions, but minimal-siphon enumeration is not exposed as an output.
[c] `crnpy` supplies Gröbner-basis, elementary-mode, and absolute-concentration-robustness machinery rather than a bistability search routine.
[d] `MØD` provides an integer hyperflow model for pathway queries rather than a firing-sequence certificate over a finite marking.
[e] ODE simulation is provided in support of `CRNT4SBML`'s bistability analysis.
[f] `MolFoundry` solves integer hyperflow models over a derivation graph (Gurobi or SMT) and returns firing orders that realize a flow from its inflow marking.
[g] Rule-based stochastic simulation with on-demand network expansion.

# 4 Conclusions

The CRN submodule of `SynKit` applies exact conservation analysis, deficiency theory, and firing certificates to curated and rule-generated reactions alike. Reference checks and an external `SBML` comparison support its structural computations, while metabolic and formose examples show how to interpret the outputs. The controlled glycolysis experiment demonstrates why preserving reactant and product incidence matters even when net stoichiometry is unchanged. Together, the analyses identify conserved metabolite pools and test whether specified substrate amounts support a proposed reaction sequence. Their interpretation remains conditional on the biochemical species and reactions included in the model.

# Availability and requirements

**Project name:** `SynKit` (CRN submodule)

**Project home page:** github.com/TieuLongPhan/SynKit

**Software version:** `SynKit` 1.6.5 [44]

**Operating system(s):** Platform independent. The test suite runs on Linux, macOS, and Windows, and the reported analyses were run on Linux

**Programming language:** Python, C++17 (analysis library), and C++20 (rule expander)

**Other requirements:** Python 3.10 or higher, `NetworkX` 3.3 or higher, `NumPy`, `SciPy`, `RDKit`, a C++17 compiler, and Boost headers. Native rule expansion additionally requires CMake, a C++20 compiler, and an RDKit C++ installation

**License:** MIT

**Any restrictions to use by non-academics:** None

## List of abbreviations

ACR: absolute concentration robustness. CRN: chemical reaction network. CRNT: chemical reaction network theory. DPO: double pushout. EMP: Embden-Meyerhof-Parnas. `KEGG`: Kyoto Encyclopedia of Genes and Genomes. `SBML`: Systems Biology Markup Language. `SMARTS`: SMILES arbitrary target specification. TCA: tricarboxylic acid. WR: weakly reversible.

## Declarations

### Ethics approval and consent to participate

Not applicable.

### Consent for publication

Not applicable.

### Availability of data and materials

The CRN submodule is distributed with `SynKit`. Source code is available on GitHub at github.com/TieuLongPhan/SynKit, including the v1.6.5 release [44]. Installation is available via PyPI at pypi.org/project/synkit/, and documentation is hosted at tieulongphan.github.io/SynKit/. The earlier v1.6.2 release is archived on Zenodo [45]. `SynKit` is released under the MIT License. The `SynKit` numerical results and timings reported here were regenerated on 6 November 2026 using the version 1.6.5 source checkout (commit `5951b401b161`) and the script `analysis/scripts/regenerate_evidence.py`, which runs every study, the external comparison, and the structural cross-checks, and installs the reports read by the manuscript. The accompanying JSON evidence records the analysis environment, the hashes of all CRN source files, native build identifiers for the analysis library and the rule expander, and input data. The comparison with `CRNT4SBML` reuses its recorded outputs after verifying that all twelve exported `SBML` inputs have identical checksums and that the recomputed `SynKit` 1.6.5 quantities agree. The legacy comparator interpreter was not rerun. The `KEGG` module records underlying Figure 2c are derived from `KEGG` [1] and cached for reproducibility, so they are not covered by `SynKit`'s MIT license and remain subject to `KEGG`'s terms. The `BioModels` inputs are identified and checksum-recorded in the evidence, while `BioModels` distributes its dataset under the CC0 1.0 Public Domain Dedication.

## Use of generative AI

Claude Opus 5.5 (Anthropic) was used for language editing, journal formatting, plotting-code revisions, the native rule-expansion program and its hydrogen-count model, and the evidence regeneration script. Codex (OpenAI) assisted with the C++ implementation, execution of the reproducible analysis scripts, and regeneration of numerical results and figures. Numerical results were computed by analysis code from recorded inputs. The authors are responsible for reviewing the code, results, and manuscript.

## Competing interests

The authors declare that they have no competing interests.


## Funding

This project has received funding from the European Union's Horizon Europe Doctoral Network programme under the Marie-Skłodowska-Curie grant agreement No 101072930 (TACsy, Training Alliance for Computational systems chemistry). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.


## Authors' contributions

Author contributions are reported using the CRediT taxonomy. **T.-A.P.**: Software, Validation, Writing (original draft), Writing (review and editing). **T.-L.P.**: Conceptualization, Methodology, Software, Validation, Writing (original draft), Writing (review and editing). Both authors read and approved the final manuscript.


## Acknowledgements

Not applicable.


# References


[1] Kanehisa M, Furumichi M, Sato Y, Matsuura Y, Ishiguro-Watanabe M (2025) Kegg: biological systems database as a model of the real world. Nucleic Acids Research 53(D1):D672–D677, DOI 10.1093/nar/gkae909

[2] Ragueneau E, Gong C, Sinquin P, Sevilla C, Beavers D, Grentner A, Griss J, Hogue GF, Li NT, Matthews L, et al (2026) The reactome knowledgebase 2026. Nucleic Acids Research 54(D1):D673–D681, DOI 10.1093/nar/gkaf1223

[3] Agrawal A, Balcı H, Hanspers K, Coort SL, Martens M, Slenter DN, Ehrhart F, Digles D, Waagmeester A, Wassink I, et al (2024) Wikipathways 2024: next generation pathway database. Nucleic Acids Research 52(D1):D679–D689, DOI 10.1093/nar/gkad960

[4] Andersen JL, Flamm C, Merkle D, Stadler PF (2016) A software package for chemically inspired graph transformation. In: International conference on graph transformation, Springer, Lecture Notes in Computer Science, vol 9761, pp 73–88, DOI 10.1007/978-3-319-40530-8_5

[5] Franzese N, Groce A, Murali T, Ritz A (2019) Hypergraph-based connectivity measures for signaling pathway topologies. PLoS Computational Biology 15(10):e1007,384, DOI 10.1371/journal.pcbi.1007384

[6] Andersen JL, Banke S, Fagerberg R, Flamm C, Merkle D, Stadler PF (2023) On the realisability of chemical pathways. In: Bioinformatics Research and Applications, International Symposium on Bioinformatics Research and Applications, vol 14248, Springer, pp 409–419, DOI 10.1007/978-981-99-7074-2_32

[7] Ebrahim A, Lerman JA, Palsson BO, Hyduke DR (2013) Cobrapy: constraints-based reconstruction and analysis for python. BMC Systems Biology 7:74, DOI 10.1186/1752-0509-7-74

[8] Orth JD, Thiele I, Palsson BØ (2010) What is flux balance analysis? Nature Biotechnology 28(3):245–248, DOI 10.1038/nbt.1614

[9] Schuster S, Hilgetag C (1994) On elementary flux modes in biochemical reaction systems at steady state. Journal of Biological Systems 2(02):165–182, DOI 10.1142/S0218339094000131

[10] Reyes BC, Otero-Muras I, Shuen MT, Tartakovsky AM, Petyuk VA (2020) Crnt4sbml: a python package for the detection of bistability in biochemical reaction networks. Bioinformatics 36(12):3922–3924, DOI 10.1093/bioinformatics/btaa241

[11] Choi K, Medley JK, König M, Stocking K, Smith L, Gu S, Sauro HM (2018) Tellurium: An extensible python-based modeling environment for systems and synthetic biology. BioSystems 171:74–79, DOI 10.1016/j.biosystems.2018.07.006

[12] Welsh C, Xu J, Smith L, König M, Choi K, Sauro HM (2023) libroadrunner 2.0: a high performance sbml simulation and analysis library. Bioinformatics 39(1):btac770, DOI 10.1093/bioinformatics/btac770

[13] Zevedei-Oancea I, Schuster S (2003) Topological analysis of metabolic networks based on Petri net theory. In Silico Biology 3(3):323–345

[14] Murata T (1989) Petri nets: Properties, analysis and applications. Proceedings of the IEEE 77(4):541–580, DOI 10.1109/5.24143

[15] Hucka M, Finney A, Sauro HM, Bolouri H, Doyle JC, Kitano H, Arkin AP, Bornstein BJ, Bray D, Cornish-Bowden A, et al (2003) The systems biology markup language (sbml): a medium for representation and exchange of biochemical network models. Bioinformatics 19(4):524–531, DOI 10.1093/bioinformatics/btg015

[16] Bergmann FT, Adams R, Moodie S, Cooper J, Glont M, Golebiewski M, Hucka M, Laibe C, Miller AK, Nickerson DP, et al (2014) Combine archive and omex format: one file to share all information to reproduce a modeling project. BMC Bioinformatics 15(1):369, DOI 10.1186/s12859-014-0369-z

[17] Wrzodek C, Dräger A, Zell A (2011) Keggtranslator: visualizing and converting the kegg pathway database to various formats. Bioinformatics 27(16):2314–2315, DOI 10.1093/bioinformatics/btr377

[18] Phan TL, González Laffitte ME, Weinbauer K, Merkle D, Andersen JL, Fagerberg R, Gatter T, Stadler PF (2025) Synkit: A graph-based python framework for rule-based reaction modeling and analysis. Journal of Chemical Information and Modeling 65(24):13,012–13,019, DOI 10.1021/acs.jcim.5c02123

[19] Müller S, Flamm C, Stadler PF (2022) What makes a reaction network "chemical"? Journal of Cheminformatics 14(1):63, DOI 10.1186/s13321-022-00621-8

[20] Shervashidze N, Schweitzer P, Van Leeuwen EJ, Mehlhorn K, Borgwardt KM (2011) Weisfeiler-lehman graph kernels. Journal of Machine Learning Research 12:2539–2561

[21] Junttila T, Kaski P (2007) Engineering an efficient canonical labeling tool for large and sparse graphs. In: 2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX), SIAM, pp 135–149

[22] McKay BD, Piperno A (2014) Practical graph isomorphism, ii. Journal of Symbolic Computation 60:94–112, DOI 10.1016/j.jsc.2013.09.003

[23] Colom JM, Silva M (1991) Convex geometry and semiflows in P/T nets: A comparative study of algorithms for computation of minimal p-semiflows. In: Advances in Petri Nets 1990, Lecture Notes in Computer Science, vol 483, Springer, pp 79–112, DOI 10.1007/3-540-53863-1_22

[24] Famili I, Palsson BO (2003) The convex basis of the left null space of the stoichiometric matrix leads to the definition of metabolically meaningful pools. Biophysical Journal 85(1):16–26, DOI 10.1016/S0006-3495(03)74450-6

[25] Horn F, Jackson R (1972) General mass action kinetics. Archive for rational mechanics and analysis 47(2):81–116, DOI 10.1007/BF00251225

[26] Feinberg M (1987) Chemical reaction network structure and the stability of complex isothermal reactors—i. the deficiency zero and deficiency one theorems. Chemical Engineering Science 42(10):2229–2268, DOI 10.1016/0009-2509(87)80099-4

[27] Feinberg M (1995) The existence and uniqueness of steady states for a class of chemical reaction networks. Archive for Rational Mechanics and Analysis 132(4):311–370, DOI 10.1007/BF00375614

[28] Gunawardena J (2003) Chemical reaction network theory for in-silico biologists

[29] Craciun G, Feinberg M (2005) Multiple equilibria in complex chemical reaction networks: I. the injectivity property. SIAM Journal on Applied Mathematics 65(5):1526–1546, DOI 10.1137/S0036139904440278

[30] Shinar G, Feinberg M (2012) Concordant chemical reaction networks. Mathematical biosciences 240(2):92–113, DOI 10.1016/j.mbs.2012.05.004

[31] Mincheva M, Roussel MR (2007) Graph-theoretic methods for the analysis of chemical and biochemical networks. i. multistability and oscillations in ordinary differential equation models. Journal of Mathematical Biology 55(1):61–86, DOI 10.1007/s00285-007-0099-1

[32] Cordone R, Ferrarini L, Piroddi L (2005) Enumeration algorithms for minimal siphons in Petri nets based on place constraints. IEEE Transactions on Systems, Man, and Cybernetics – Part A 35(6):844–854, DOI 10.1109/TSMCA.2005.853504

[33] Angeli D, De Leenheer P, Sontag ED (2007) A petri net approach to the study of persistence in chemical reaction networks. Mathematical Biosciences 210(2):598–618, DOI 10.1016/j.mbs.2007.07.003

[34] Andersen JL, Flamm C, Merkle D, Stadler PF (2014) Generic strategies for chemical space exploration. International Journal of Computational Biology and Drug Design 7(2/3):225–258, DOI 10.1504/IJCBDD.2014.061649

[35] Holzschuh R, Flamm C, Merkle D, Stadler PF (2026) Half-edge graphs, Lewis formulas, and electron pushing. ChemRxiv preprint, DOI 10.26434/chemrxiv.15006001/v1

[36] Wang L, Sontag ED (2008) On the number of steady states in a multiple futile cycle. Journal of Mathematical Biology 57(1):29–52, DOI 10.1007/s00285-007-0145-z

[37] Edelstein BB (1970) Biochemical model with multiple steady states and hysteresis. Journal of Theoretical Biology 29(1):57–62, DOI 10.1016/0022-5193(70)90118-9

[38] Shinar G, Feinberg M (2010) Structural sources of robustness in biochemical reaction networks. Science 327(5971):1389–1391, DOI 10.1126/science.1183372

[39] Malik-Sheriff RS, Glont M, Nguyen TVN, Tiwari K, Roberts MG, Xavier A, Vu MT, Men J, Maire M, Kananathan S, Fairbanks EL, Meyer JP, Arankalle C, Varusai TM, Knight-Schrijver V, Li L, Due nas-Roca C, Dass G, Keating SM, Park YM, Buso N, Rodriguez N, Hucka M, Hermjakob H (2020) Biomodels—15 years of sharing computational models in life science. Nucleic Acids Research 48(D1):D407–D415, DOI 10.1093/nar/gkz1055

[40] Butlerow A (1861) Bildung einer zuckerartigen substanz durch synthese. Justus Liebigs Annalen der Chemie 120(3):295–298, DOI 10.1002/jlac.18611200308

[41] Donnell P, Banaji M, Marginean A, Pantea C (2014) CoNtRol: an open source framework for the analysis of chemical reaction networks. Bioinformatics 30(11):1633–1634, DOI 10.1093/bioinformatics/btu063

[42] Tonello E (2016) crnpy: a Python library for the analysis of chemical reaction networks. Zenodo DOI 10.5281/zenodo.30736, https://github.com/etonello/crnpy

[43] Friedrichs M (2026) MolFoundry: rule-based graph transformation for chemistry, version 0.1. https://py.molfoundry.de/docs/, software documentation, accessed 5 November 2026

[44] Phan TL, Weinbauer K (2026) SynKit, version 1.6.5. https://github.com/TieuLongPhan/SynKit/releases/tag/v1.6.5, released 30 October 2026

[45] Phan TL, Weinbauer K (2026) SynKit, version 1.6.2. DOI 10.5281/zenodo.21991269, released 18 August 2026. Earlier software archive